MAMBA

MCP-enabled Autonomous MAVLink-M Bridge for AI



Team:

Daniel Stein (CS), Jake Lawrence (CS), Rachel Miller (CS)

Customer:

Maj Zach “MIAMI” Laser, Defense Autonomous Warfare Group (DAWG)
Cell: (757) 642-8659
Email: zachary.s.laser.mil@socom.mil

Customer’s initials: _____ Date initialed: _____
“I have reviewed the Team’s Capstone Proposal, and understand that the team will be following the Agile Software Development approach. I may ask for the addition, removal, or modification of any functional or non-functional requirements and/or acceptance test cases at any time as the Capstone Project progresses.”

1. Abstract....3
2. Project Introduction....3
2.1 Current Situation:....3
2.2 The Problem:....3
2.2 The Solution:....3
2.2 Core Deliverables:....4
3. Market Research....4
3.1 Similar Products....4
3.2 Literature Review....5
3.3 Existing Resources....6
4. Design & Modeling....7
4.1 Layered Architecture....7
5. Initial Release Plan....9
6. Team....10
6.1 Midshipmen....10
6.2 SMEs & Mentors....10
7. Risk Management....11
8. GFI/GFE....11
9. Appendix: Feature List....12

1. Abstract

MAMBA is a model-agnostic interface that allows an operator to communicate high-level intent to an AI agent capable of interacting with multiple unmanned systems and payloads via a Model Context Protocol (MCP) Server. The system incorporates explicit safety constraints, authorization requirements, and deterministic command sequencing to ensure that requested actions are valid for the current state of both the aircraft and payload. MAMBA follows Anthropic's MCP standard and outputs both MAVLink and MAVLink-M frames for use with compatible UxS.

2. Project Introduction

Our goal is to implement an MCP Server to bridge an AI Agent with MAVLink/MAVLink-M-enabled UxSs and their payloads.

2.1 Current Situation:

Right now, most UxS (unmanned systems) that special operations teams operate use an open-source, lightweight messaging protocol known as MAVLink. This protocol allows a single operator to control a single UxS via a GCS (ground control station). The GCS communicates actions to the autopiloting software running on the onboard flight controller and receives telemetry and drone health information, all through MAVLink. If the drone encounters an error, the operator can take immediate manual control.

2.2 The Problem:

When a mission commander wants to use multiple drones to conduct different parts of a single mission, they need one operator for each drone, and communication and coordination must happen between all of the operators. As drone warfare progresses and commanders need the ability to scale up, having a single fully trained operator for each UxS becomes a manning and training issue. This is especially true as missions begin to span across multiple domains requiring a mix of UAS, USV, and UUV. Although these drones can operate through a GCS using MAVLink, that still requires extensive communication and coordination between the operators. This acts as the opposite of a force multiplier, limiting scalability and preventing a team from utilizing their drones and operators to full effect.

2.2 The Solution:

We will implement an MCP server that exposes drone and payload capabilities as tools that an AI Agent can use to translate natural language intent into a series of actionable MAVLink messages for the hardware to execute. Our MCP server will incorporate explicit safety

constraints, authorization requirements, and deterministic command sequencing to ensure that requested actions are valid for the current state of both the aircraft and payload.

Additionally, MAMBA will support simultaneous connections to multiple UAS through a single MCP server and AI Agent. The system will maintain independent vehicle connections, telemetry, and safety states, allowing the Agent to identify and direct commands to specific vehicles based on operator intent. This will enable a single operator to interact with and manage multiple unmanned systems through one interface, reducing the need for a dedicated operator for each vehicle.

2.2 Core Deliverables:

Our core deliverables are:

1. MCP Server with discoverable tools for both MAVLink and MAVLink-M commands and telemetry
2. Implemented state machines for each supported drone platform and payload
3. A simulation test platform for benchmarking
4. Data on the effectiveness and viability of the MCP server implementation in simulation and with hardware

3. Market Research

Products already exist for using AI to control ordinary MAVLink drones, and broader systems exist for connecting AI to robots. Many of these systems are developed to the specifications of particular hardware, which limits their scalability. In addition, this research found no public MCP designed specifically around MAVLink-M. The proposed project therefore addresses a narrower interoperability problem while simultaneously building a more robust general MAVLink MCP than is currently available in the open-source community.

3.1 Similar Products

DroneServer

An open-source MCP server made by Peter J. Burke. It connects AI assistants to PX4 and ArduPilot drones through MAVSDK/MAVLink. It is primarily a proof of concept for Burke's research in the field, which is extremely relevant to this project.

DroneServer is a general-purpose drone MCP, but it focuses on normal flight-control functions. The proposed capstone focuses on MAVLink-M, which adds coordination and command-and-control information that is outside the main purpose of ordinary MAVSDK flight tools. Additionally, it only supports control of one UxS, and it does not implement fire control safety features. One of our client's key concerns was that existing solutions, such as DroneServer, were hastily produced with excessive use of AI-generated code. This is evidenced by many commits to Dronecode's GitHub, which reference

Claude as a co-author. Despite these limitations, DroneServer is the most feature-complete open-source MCP for MAVLink, and therefore serves as a key reference point for MAMBA.

Reference: https://github.com/PeterJBurke/droneserver

MAVLinkMCP

An open-source MCP server developed by the anonymous GitHub user ion-g-ion. MAVLinkMCP connects AI agents to MAVLink-enabled vehicles through MAVSDK and exposes a limited set of conventional flight-control functions, including basic navigation, telemetry, mission planning, and preflight checks. It provides a useful example of how MAVLink functionality can be abstracted into MCP tools for use by an AI agent.

MAVLinkMCP is narrower in scope than MAMBA. It focuses on conventional single-vehicle flight operations and does not implement MAVLink-M, multi-UxS command and control, or military payload and fire-control functions. MAMBA instead seeks to provide broader MAVLink support while extending the MCP architecture to MAVLink-M and incorporating safety mechanisms appropriate for AI-mediated military UxS control.

Reference: https://github.com/ion-g-ion/MAVLinkMCP

ROS MCP Server

An open-source project from the Robot MCP community that connects AI assistants to robots using ROS and ROS 2. It can read robot state and call robot functions without requiring a separate MCP implementation for every robot type. It is intended for robotics developers using ROS.

This is broader than a drone-specific MCP and can potentially reach PX4-based aircraft through ROS 2. ROS is a completely different standard than MAVLink, but may achieve a similar effect. ROS operates at a higher layer of abstraction than MAVLink, requiring more overhead and is therefore unsuited to military applications. Additionally, it does not provide MAVLink-M-specific concepts or ensure that an AI request is converted into the message format expected by MAVLink-M systems.

Reference: https://github.com/robotmcp/ros-mcp-server

3.2 Literature Review

Vemprala et al., "ChatGPT for Robotics: Design Principles and Model Abilities" (Microsoft Research, 2023)

This paper studies how a language model can control robots, including drones, from natural-language instructions. The authors show that an AI model can reason about a task and select robot functions rather than requiring the user to write robot code. The system gives the language model a high-level library of robot functions and uses prompting and dialogue to translate user intent into calls to those functions. The experiments include aerial navigation and simulated robotics tasks.

This supports the design of an MCP layer that gives an AI model a limited set of understandable actions instead of exposing low-level protocol details. It does not address MAVLink-M, MCP, or the newer defense-oriented coordination messages required by this capstone.

Reference:
https://www.microsoft.com/en-us/research/publication/chatgpt-for-robotics-design-principles-and-model-abilities/

Lim et al., "Taking Flight with Dialogue: Enabling Natural Language Control for PX4-based Drone Agent" (2025)

This paper presents an open-source system that lets users control a PX4 drone with natural language. The authors test the system both in simulation and on a custom quadcopter. Their design combines PX4 flight control, ROS 2, and locally hosted language and vision models through Ollama. The language model produces commands that are then passed through the robotics software stack to the aircraft.

The work is close to our capstone because it demonstrates an end-to-end natural-language-to-drone pipeline using common open-source drone software. It focuses on controlling one drone through PX4/ROS 2 and does not provide an MCP interface for MAVLink-M coordination, tasking, or command-system interoperability.

Reference: https://arxiv.org/abs/2506.07509

Ramos-Silva and Burke, "A Universal Large Language Model - Drone Command and Control Interface" (2026)

This paper describes DroneServer, the MCP system discussed above, and tests natural-language control of both real and simulated MAVLink drones. It shows that a language model can use one common interface for flight commands and drone status. A Python MCP server exposes selected MAVSDK functions as higher-level tools, while MAVLink handles communication with PX4 or ArduPilot. The authors also add status and timing logic after testing showed that language models could issue flight actions too quickly or out of sequence.

This is the closest published work to the proposed capstone because it validates MCP as a bridge between language models and MAVLink drones. However, it targets general MAVLink/MAVSDK flight control rather than MAVLink-M coordination and information-sharing messages, so it does not meet the capstone's MAVLink-M-specific need.

Reference: https://arxiv.org/html/2601.15486v2

3.3 Existing Resources

Model Context Protocol

An open standard promulgated by Anthropic to connect AI models to external data sources, tools, and environments. It communicates with the AI model via structured JSON. It is intended to eliminate the need for custom integrations by providing a universal system for how AI models retrieve context and execute tasks.

Reference: https://modelcontextprotocol.io/

MAVLink

A lightweight messaging protocol designed for communication between UxS, onboard components and GCS. A compact binary format, it minimizes overhead to transmit telemetry, waypoints and control commands over radio or serial links. It is the standard language across major open-source drone autopilot flight control platforms like PX4 and ArduPilot.

Reference: https://mavlink.io/en/

MAVLink-M

A MAVLink dialect extension that allows for greater coordination and situational-awareness messaging for military drones. It outlines new MAVLink frames for a variety of mission types, including intelligence, surveillance, and reconnaissance (ISR), non-kinetic and kinetic effects. No current hardware can accept MAVLink-M messages.

Reference: https://github.com/Dronecode/mavlink-military

4. Design & Modeling

4.1 Layered Architecture

The MCP Server will use a layered architecture, as shown in Figure 4.1.1. The proposed architecture separates high-level operator intent, AI reasoning, safety enforcement, and vehicle execution into distinct layers. A human operator provides a natural-language command to an LLM-based AI agent. The agent interprets the request and selects the appropriate capabilities exposed by the MCP server.

Requested actions are passed through a dedicated safety layer before they are allowed to reach the drone. This layer provides deterministic enforcement of system constraints such as human-in-the-loop authorization, command sequencing, rate limits, and state machine transitions. This layer will implement an individual state machine for each supported platform and payload.

The safety layer will enforce a defined operating envelope for each supported vehicle and payload. The operating envelope and required state definitions will be provided to the team as project requirements in coordination with the customer and subject matter experts. These requirements will establish the platform-specific permissible operating conditions and restrictions that MAMBA must enforce. Our responsibility is to implement

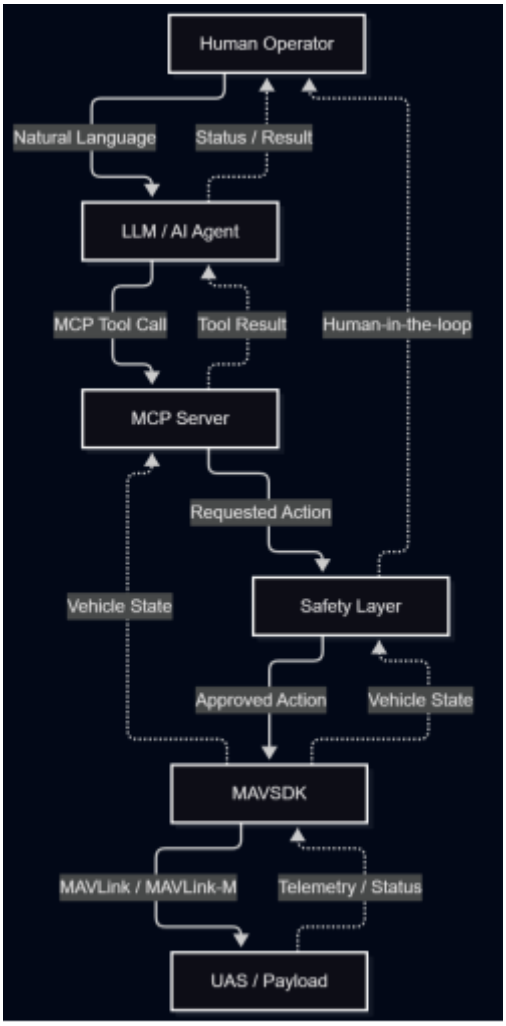


Figure 4.1.1 Architecture

these requirements through deterministic finite state machines (FSMs), rather than relying on the AI Agent to interpret or enforce safety constraints.

Because MAVLink and MAVLink-M are designed to be platform agnostic, the safety layer must also support new vehicle and payload types without requiring significant changes to MAMBA's core architecture. To accomplish this, we will develop a modular, configuration-driven framework for defining platform-specific operating envelopes and state machines. Configuration files will specify the states, valid transitions, environmental limitations, and other safety constraints for each supported vehicle or payload. This approach will allow new platforms to be integrated by defining and validating their safety requirements within the framework rather than hard-coding platform-specific logic into the safety layer.

A key design consideration will be determining how these finite state machines integrate into the overall MAMBA architecture and communicate with the MCP server. Each connected vehicle and payload will have its own associated state representation, maintained using incoming telemetry and system information. When the Agent requests an operation through an MCP tool, the safety layer will evaluate the request against the applicable vehicle and payload states, operating envelope, and authorization requirements before allowing the MCP server to forward the command for execution. The safety layer will return an approval or rejection to the MCP server, along with the appropriate status or explanation. If required state information is unavailable, outdated, or inconsistent, the safety layer will reject the operation rather than assume that it is safe.

Once an action has been approved by the safety layer, it is forwarded to MAVSDK. MAVSDK provides the software interface used to communicate with MAVLink-enabled vehicles and translates higher-level operations into the appropriate MAVLink interactions. Currently, MAVSDK does not have support for MAVLink-M commands, so part of the capstone will be stubbing out the implementations of these commands for testing, while industry develops them professionally.

Operator intent and drone commands flow down the layers while information flows in the opposite direction. The UAS and payload return telemetry, state information, command acknowledgments, and status updates through MAVLink and MAVLink-M. MAVSDK exposes this information to both the MCP server and the safety layer. The MCP server converts vehicle state into structured JSON tool results that can be returned to the AI agent, while the safety layer uses current system state to evaluate whether future commands are valid. The AI agent can then summarize this information for the operator in natural language.

The safety layer also maintains a direct human-in-the-loop path for actions that require explicit operator approval. When such an action is requested, the system can pause execution until authorization is received from the operator. This ensures that safety-critical or restricted actions cannot be executed solely through AI-generated tool calls

Our MCP server will be written in Python utilizing its built-in OOP principles, and the bulk of our bespoke code will be written for the MCP Server Layer and the Safety Layer. An important part of the capstone will be deciding which MAVSDK plugins to wrap in MCP tools,

and how those tools are advertised to the LLM. The fixed size of an LLM context window, especially a local one operating at the tactical edge, demonstrates a significant challenge. We will determine the exact number of tools within the MCP and how the plugins are organized with benchmarking and testing.

5. Initial Release Plan

Serial	Feature Name	Story Points (Estimate)	Delivery Date
F1	MCP Server Foundation	3	**23OCT2026**
F2	MAVSDK Drone Connection	3	**23OCT2026**
F3	MAVLink Basic Navigation and Telemetry Tools in MCP Server	5	**6NOV2026**
F4	Action Logging & Audit Trail	3	**6NOV2026**
F5	Gazebo Simulation Pipeline	3	**30NOV2026**
F6	Initial Testing	5	**30NOV2026**
F7	MAVLink Advanced Navigation and Mission Planning Tools in MCP Server	5	**6DEC2026**
F8	State Machine for UAV Implemented in Safety Layer	3	**6DEC2026**
F9	Implement sequencing and timing enforcement for the Agent in the Safety Layer	3	**20DEC2026**
F10	Stub out Human Operator interface	8	**20JAN2027**
F11	Implement Human Confirmation and Authorization through interface	5	**20JAN2027**
F12	Stub Out MAVLink-M support in MAVSDK	3	**6FEB2027**
F13	Implement Manual Override	2	**6FEB2027**

F14	Extend Gazebo w/ MAVLink-M	5	**20FEB2027**
F15	Add State Machines for Payloads in Safety Layer	5	**20FEB2027**
F16	Safety Layer Testing in Simulation	5	**5MAR2027**
F17	Building out Hardware Testing Pipeline	5	**19MAR2027**
F18	Add State Machines for USV and UGV in Safety Layer	3	**19MAR2027**
F19	Hardware Testing	5	**30MAR2027**
F20	Multi-UAS Management and Command Routing	8	**14APR2027**

6. Team

6.1 Midshipmen

MIDN 1/C Daniel Stein, USNA
MIDN 1/C Jake Lawrence, USNA
MIDN 1/C Rachel Miller, USNA

Each member of the team is a Computer Science major with multiple years of experience in programming, including relevant knowledge in data structures, networking, and machine learning.

6.2 SMEs & Mentors

Andrew Weinert, MIT LL

A member of the Homeland Protection and Air Traffic Control Division at MIT Lincoln Laboratory. Research focuses on collision avoidance for unmanned aircraft and public safety systems. Closely connected with Dronecode, the organization responsible for MAVLink-M.

Daniel Stabile, MIT LL

A software developer at MIT Lincoln Laboratory focusing on AI/ML. Previous work focuses on robotics and applications of agentic AI. Currently working with the customer on a related project which has potential to be used as a real-world test case for MAMBA.

2LT Kristian Nordby, USA, MIT LL

Military Fellow at MIT Lincoln Laboratory, alumnus of the SOCOM Ignite program. Currently researching unmanned systems.

LCDR Justin Downs, USN, CS PMI

Permanent military instructor at the Naval Academy with Agentic AI experience. Also attended SOCOM Ignite with the group.

7. Risk Management

The project faces three primary technical and operational risks. First, MAMBA depends on MAVLink-M, an open-source project that remains under active development and is therefore subject to changes that could affect our implementation. Our project mentor, Andrew Weinert, is in close communication with the MAVLink-M maintainers, who anticipate a code freeze in November. Until then, we will maintain a modular implementation that can accommodate changes to the specification with minimal rework.

Second, there is currently no mature testing environment for MAVLink-M functionality. To address this gap, we will develop our own extension to Gazebo that will simulate the relevant MAVLink-M capabilities and provide a controlled environment for integration and end-to-end testing.

Finally, testing autonomous drone-control functionality on physical hardware introduces risks to personnel, equipment, and the surrounding environment. We will mitigate these risks by conducting development and validation primarily in simulation, progressing to physical testing only after functionality has been demonstrated in the simulated environment, and maintaining human supervision and manual override capabilities during real-world testing.

8. GFI/GFE

MAMBA does not require GFI/GFE.

9. Appendix: Feature List

Feature F1: MCP Server Foundation

Prerequisites: None.

User Story: As a **drone operator**, I want my **AI Agent to be able to connect to a MAMBA Server and discover tools**.

Context: This feature establishes the base Model Context Protocol (MCP) server used by MAMBA. The server must accept connections from an MCP-capable AI Agent and expose registered tools through MCP tool discovery. At this stage, the tools may be simple test tools and do not need to communicate with a physical or simulated vehicle.

Extensions / Error States:

- If the MCP server cannot initialize, it should terminate with a useful error message.
- Invalid MCP requests should return structured errors rather than crash the server.
- Failure of an individual MCP tool should not terminate the server.
- A disconnected Agent should be able to reconnect without requiring a server restart.

Documentation: Document installation requirements, how to start the MAMBA server, the supported MCP transport, how tools are registered, and how an MCP-capable Agent connects to MAMBA.

Acceptance Test

You will need:

- MAMBA development environment.
- MCP-capable client or Agent.
- Required MCP dependencies.

Steps:

1. Start the MAMBA server.
2. Connect an MCP-compatible client.
3. Request the available tool list.
4. Execute a basic test tool.
5. Send an invalid tool request.

- The MAMBA server starts successfully.
- An MCP-capable client can connect.
- The client can discover registered tools.

- A registered test tool can be invoked.
- Invalid requests return a controlled error.
- Invalid requests do not terminate the MAMBA server.

Feature F2: MAVSDK Drone Connection

Prerequisites: F1 — MCP Server Foundation.

User Story: As a **drone operator**, I want the **ability to connect my ArduPilot enabled drone to the MCP Server**.

Context: MAMBA must establish and maintain a connection to an ArduPilot-enabled UAS using MAVSDK and MAVLink. The connection should persist between MCP tool calls rather than requiring a new vehicle connection for every request. The connection layer should expose basic vehicle availability and connection state to other MAMBA components.

Extensions / Error States:

- Vehicle is unavailable.
- MAVSDK connection times out.
- Vehicle disconnects during operation.
- Incorrect connection URI or port is supplied.
- Multiple MAVLink systems are detected.

A vehicle connection failure should not terminate the MCP server.

Documentation: Document supported MAVLink connection methods, SITL connection configuration, physical vehicle connection configuration, and expected ArduPilot configuration.

Acceptance Test

You will need:

- Running MAMBA server.
- MAVSDK.
- ArduPilot SITL or physical ArduPilot-enabled vehicle.

Steps:

1. Start the ArduPilot vehicle or SITL instance.
2. Start MAMBA using the correct connection configuration.
3. Verify that MAVSDK discovers the vehicle.

4. Verify MAMBA can identify that the vehicle is connected.
5. Disconnect the vehicle.

- MAVSDK successfully discovers the vehicle.
- MAMBA retains a persistent connection to the vehicle.
- Vehicle connection state can be determined.
- Loss of the vehicle connection is detected.
- Vehicle disconnection does not crash MAMBA.

Feature F3: MAVLink Basic Navigation and Telemetry Tools in MCP Server

Prerequisites: F1 — MCP Server Foundation; F2 — MAVSDK Drone Connection.

User Story: As a **drone operator**, I want the **ability to obtain the health of my drone as well as give it basic commands such as arm, takeoff, and land**.

Context: MAMBA should expose basic UAS capabilities as semantic MCP tools rather than requiring the Agent to understand MAVLink messages directly. Initial capabilities should include vehicle health, position and altitude, battery information, arm, disarm, takeoff, land, and return-to-launch.

Extensions / Error States:

- Vehicle is disconnected.
- Vehicle is unhealthy.
- MAVSDK rejects a command.
- Required telemetry is unavailable.
- Invalid tool parameters are provided.
- The vehicle cannot perform the requested operation in its current state.

MAVSDK errors should be converted into useful MCP responses for the Agent.

Documentation: Document each MCP tool, including its purpose, parameters, return value, associated MAVSDK capability, and known limitations.

Acceptance Test

You will need:

- MAMBA server.
- Connected ArduPilot SITL vehicle.
- MCP-capable Agent or test client.

Steps:

1. Connect the Agent to MAMBA.
2. Request vehicle health.
3. Request vehicle position and battery status.
4. Arm the vehicle through an MCP tool.
5. Command takeoff.
6. Request updated telemetry.
7. Command the vehicle to land.

- Health information is returned correctly.
- Telemetry is available through MCP tools.
- Arm executes successfully when permitted.
- Takeoff executes successfully.
- Land executes successfully.
- MAVSDK failures are returned as structured tool results.

Feature F4: Action Logging & Audit Trail

Prerequisites: F1 — MCP Server Foundation; F3 — Basic Navigation and Telemetry Tools.

User Story: As a **drone operator**, I want the **ability to see a record of every tool call my Agent makes as well as the outcome/response by the drone and the MCP Server**.

Context: MAMBA should record a chronological audit trail that allows an operator or developer to reconstruct what the Agent requested, how the system processed the request, and what result was returned. Where applicable, records should include timestamp, tool name, tool arguments, Agent/session identity, safety decision, MAVSDK result, vehicle response, and final MCP result.

Extensions / Error States:

- Log destination is unavailable.
- A log entry cannot be written.
- Tool execution fails after a request is logged.
- Sensitive authorization information must not be exposed unnecessarily.

Logging failures should be detectable and should not silently discard audit information.

Documentation: Document the log format, storage location, recorded fields, and how logs can be reviewed.

Acceptance Test

You will need:

- Running MAMBA server.
- Connected simulated UAS.
- Access to generated logs.

Steps:

1. Execute several MCP tools.
2. Execute at least one request that fails.
3. Review the audit trail.

- Every Agent tool call produces an audit entry.
- Tool arguments are recorded where appropriate.
- Vehicle or MAVSDK responses are recorded.
- Failed and rejected actions remain visible.
- Entries include timestamps.
- The command sequence can be reconstructed from the audit trail.

Feature F5: Gazebo Simulation Pipeline

Prerequisites: F2 — MAVSDK Drone Connection; F3 — Basic Navigation and Telemetry Tools.

User Story: As a **developer**, I want to **be able to operate a simulated quadcopter in Gazebo using an Agent equipped with MAMBA**.

Context: The simulation pipeline provides a repeatable environment for developing and testing MAMBA without risking physical hardware. The simulated quadcopter should communicate with MAVSDK through the same MAVLink interface expected from a physical ArduPilot-enabled aircraft.

Extensions / Error States:

- Gazebo fails to start.
- ArduPilot SITL fails to start.
- MAVLink communication cannot be established.
- Network ports conflict.
- Simulation unexpectedly terminates.

Documentation: Provide required Gazebo/SITL dependencies, setup instructions, startup command or script, shutdown procedure, and known simulation limitations.

Acceptance Test

You will need:

- Gazebo.
- ArduPilot SITL.
- MAMBA.
- MCP-compatible Agent.

Steps:

1. Start the documented simulation pipeline.
2. Verify that a quadcopter appears in Gazebo.
3. Start MAMBA.
4. Connect an Agent to MAMBA.
5. Request telemetry.
6. Arm, take off, and land the simulated vehicle.

- Gazebo starts successfully.
- MAVSDK connects to the simulated quadcopter.
- MAMBA receives telemetry.
- Basic MCP commands control the simulated vehicle.
- The environment can be restarted using the documented procedure.

Feature F6: Initial Testing

Prerequisites: F1 through F5.

User Story: As a **developer**, I want to **verify the Agent's ability to use Basic Navigation Tools in simulated test cases**.

Context: Initial testing should establish that an MCP-capable Agent can discover the basic tools provided by MAMBA, choose appropriate tools, and successfully operate a simulated UAS. Tests should include both successful and unsuccessful operations.

Extensions / Error States: Tests should include invalid MCP tool arguments, vehicle disconnection, unhealthy vehicle state, MAVSDK command failure, and invalid tool requests.

Documentation: Document how to run the tests, the test environment configuration, expected results, and known limitations.

Acceptance Test

You will need:

- MAMBA.
- Gazebo/SITL environment.
- MCP-capable Agent.
- Initial test suite.

Steps:

1. Start the simulation environment.
2. Start MAMBA.
3. Connect the Agent.
4. Execute the defined basic navigation test scenarios.
5. Execute the defined failure scenarios.
6. Review test output and logs.

- Agent discovers the required basic tools.
- Agent can obtain telemetry.
- Agent can execute basic navigation commands.
- Expected failure conditions are handled correctly.
- All required initial tests pass.

Feature F7: State Machine for UAV Implemented in Safety Layer

Prerequisites: F3 — Basic Navigation and Telemetry Tools; F4 — Action Logging & Audit Trail; F5 — Gazebo Simulation Pipeline.

User Story: As a **drone operator**, I want to **ensure that my Agent is only able to use the tools that can currently be supported by the drone in its current state**.

Context: MAMBA's safety layer should maintain a deterministic representation of vehicle state based on telemetry received from the UAS. Representative states may include disconnected, disarmed, armed, airborne, landing, and emergency/error. The safety layer should determine whether an Agent-requested operation is valid before allowing the operation to reach MAVSDK.

Extensions / Error States:

- Vehicle state cannot be determined.
- State information is stale.
- Agent requests an invalid state transition.
- Vehicle telemetry conflicts with the expected state.
- Vehicle disconnects during a transition.

When the state cannot be confidently determined, the system should default to the safer behavior.

Documentation: Provide a UAV state diagram defining states, valid transitions, invalid transitions, and MCP tools permitted in each state.

Acceptance Test

You will need:

- MAMBA.
- Gazebo/SITL.
- Defined UAV state machine.
- MCP test client.

Steps:

1. Place the simulated vehicle in a known state.
2. Request a valid operation.
3. Verify that the command is permitted.
4. Request an invalid operation for the current state.
5. Inspect the audit trail.

- Valid actions are permitted.
- Invalid actions are rejected.
- A rejected command does not reach MAVSDK.
- UAV state reflects vehicle telemetry.
- State-related rejections are logged.
- The Agent receives a useful explanation of the rejection.

Feature F8: MAVLink Advanced Navigation and Mission Planning Tools in MCP Server

Prerequisites: F3 — Basic Navigation and Telemetry Tools; F5 — Gazebo Simulation Pipeline; F7 — UAV State Machine.

User Story: As a **drone operator**, I want the **ability to describe an advanced mission to the Agent and it can utilize the necessary tools to complete the mission (go_to_waypoint, loiter, scan_grid, etc)**.

Context: This feature expands MAMBA beyond individual basic actions into semantic tools that support multi-step autonomous missions. Potential capabilities include `go_to_waypoint`, `loiter`, `scan_grid`, mission upload, mission start, mission pause/resume, mission cancellation, and mission progress.

Extensions / Error States:

- Invalid waypoint or location.
- Mission cannot be uploaded.
- Vehicle cannot enter the required flight state.
- Mission is interrupted.
- Mission conflicts with the UAV state machine.
- Required telemetry becomes unavailable.

Documentation: Document advanced tools, parameters, outputs, and representative mission workflows.

Acceptance Test

You will need:

- MAMBA.
- Gazebo/SITL environment.
- MCP-capable Agent.
- Defined advanced mission scenarios.

Steps:

1. Provide the Agent a natural-language mission containing multiple actions.
2. Observe the tools selected by the Agent.
3. Execute the mission in simulation.
4. Monitor vehicle progress.
5. Execute a mission containing an invalid request.

- Agent discovers advanced mission tools.
- Agent selects appropriate tools for the requested mission.
- Valid mission actions execute successfully.
- Mission progress can be monitored.
- Invalid mission operations are rejected or reported correctly.

Feature F9: Implement Sequencing and Timing Enforcement for the Agent in the Safety Layer

Prerequisites: F7 — UAV State Machine; F8 — Advanced Navigation and Mission Planning Tools.

User Story: As a **drone operator**, I want my **drone to be constrained to only make MCP calls appropriate for the current status of the drone as determined by telemetry information**.

Context: The Agent may request several MCP tools sequentially or concurrently. The safety layer must remain authoritative over whether and when a requested command is allowed to execute. Controls may include per-vehicle command queues, execution locks, rate limits, minimum command intervals, completion checks, and state-transition sequencing.

Extensions / Error States:

- Multiple commands arrive simultaneously.
- Agent exceeds an allowed tool-call rate.
- Previous operation has not completed.
- Required telemetry is stale.
- State transition times out.
- Conflicting operations are requested.

Documentation: Document sequencing rules, rate limits, timing restrictions, and queueing/rejection behavior.

Acceptance Test

You will need:

- MAMBA.
- Simulated UAS.
- MCP client capable of issuing simultaneous calls.

Steps:

1. Send a valid sequence of commands.
2. Send multiple conflicting commands concurrently.
3. Exceed a configured tool rate limit.
4. Issue a command before a required previous transition completes.
- Valid command sequences execute correctly.
- Conflicting state-changing operations do not execute concurrently.
- Rate limits are enforced.
- Required timing rules are enforced.
- Violations produce structured feedback.
- Safety behavior remains deterministic.

Feature F10: Stub Out Human Operator Interface

Prerequisites: F1 — MCP Server Foundation; F3 — Basic Telemetry Tools.

User Story: As a **drone operator**, I want to **be able to receive telemetry information and pass natural language intents to my Agent**.

Context: The initial operator interface establishes a user-facing path between the operator, Agent, and MAMBA. The first implementation does not need to provide a complete ground-control station. It should primarily support natural-language input and presentation of important system and telemetry information.

UI Changes: The interface should provide a natural-language command input, Agent response/output, vehicle connection status, basic telemetry display, and an area for system notifications.

Extensions / Error States:

- Agent unavailable.
- MAMBA unavailable.
- Vehicle disconnected.
- Telemetry unavailable.
- Natural-language request cannot be submitted.

The interface should clearly communicate these conditions to the operator.

Documentation: Document how to start, connect to, and use the operator interface.

Acceptance Test

You will need:

- Human operator interface.
- MAMBA.
- Agent.
- Simulated vehicle.

Steps:

1. Launch the operator interface.
2. Connect to the Agent/MAMBA backend.
3. Verify telemetry is displayed.
4. Enter a natural-language request.
5. Observe the Agent response.

- Interface loads successfully.
- Operator can submit natural-language intent.
- Agent response is displayed.
- Vehicle connection status is visible.
- Basic telemetry is visible.
- Backend failures are clearly communicated.

Feature F11: Implement Human Confirmation and Authorization Through Interface

Prerequisites: F7 — UAV State Machine; F9 — Sequencing and Timing Enforcement; F10 — Human Operator Interface.

User Story: As a **drone operator**, I want my **Agent to request authorization before making sensitive tool calls**.

Context: MAMBA should classify selected operations as requiring explicit human approval. The safety layer—not the Agent itself—should enforce this requirement. The Agent may request the operation, but the protected action cannot proceed until valid operator authorization is received. Authorization should apply only to the particular operation and parameters presented to the operator.

UI Changes: The interface should display the requested sensitive action, relevant parameters, affected vehicle or payload, approve and deny controls, and request status.

Extensions / Error States:

- Operator denies the request.
- Approval expires.
- Approval token is reused.
- Parameters change after authorization.
- Unauthorized user attempts approval.
- Communication is lost while approval is pending.

No sensitive command should execute under these conditions.

Documentation: Document which actions require authorization, the authorization workflow, approval lifetime, and approval invalidation behavior.

Acceptance Test

You will need:

- Operator interface.
- MAMBA.
- Agent.
- Simulated UAS.
- A tool designated as requiring human approval.

Steps:

1. Have the Agent request a protected operation.
2. Verify the operator receives an authorization request.
3. Deny the request.
4. Verify the action does not execute.
5. Repeat the request.
6. Approve it.
7. Attempt to reuse the same authorization.

- Protected actions stop before execution.
- Operator is shown the requested action.
- Denial prevents execution.
- Approval permits the intended action.
- Approval cannot be reused for another operation.
- Authorization activity is recorded in the audit trail.

Feature F12: Stub Out MAVLink-M Support in MAVSDK

Prerequisites: F2 — MAVSDK Drone Connection; F3 — Existing MAVLink/MAVSDK integration.

User Story: As a **developer**, I want to **use MAVSDK to make MAVLink-M calls which are not natively supported**.

Context: MAVLink-M payload messages may not be represented by existing high-level MAVSDK plugins. MAMBA therefore requires an abstraction for sending and receiving MAVLink-M messages through the MAVSDK integration layer. The implementation should prevent higher-level MCP tools from needing to construct transport-level MAVLink messages directly.

Extensions / Error States:

- MAVLink-M dialect cannot be loaded.
- Unsupported message is requested.
- Target component is unavailable.

- Message encoding or decoding fails.
- Payload does not respond.

Documentation: Document the supported MAVLink-M dialect/version, MAVSDK extension mechanism, message-generation process, and how new MAVLink-M capabilities are added.

Open Questions / Assumptions:

- Determine whether MAVLink-M interaction will use MAVSDK passthrough/direct message interfaces or a custom MAVSDK plugin.
- Determine how MAVLink-M payload components will be discovered and addressed.

Acceptance Test

You will need:

- MAMBA development environment.
- MAVSDK.
- MAVLink-M dialect definition.
- Test endpoint or message test harness.

Steps:

1. Build MAMBA with MAVLink-M support enabled.
2. Initialize the MAVLink-M interface.
3. Construct and send a representative MAVLink-M message.
4. Receive or decode a representative response.
5. Request an unsupported message.

- MAVLink-M support builds successfully.
- A supported MAVLink-M message can be generated.
- A supported message can be transmitted through the MAVSDK layer.
- Responses can be decoded.
- Unsupported operations return controlled errors.

Feature F13: Implement Manual Override

Prerequisites: F7 — UAV State Machine; F9 — Sequencing and Timing Enforcement; F10 — Human Operator Interface.

User Story: As a **drone operator**, I want the **ability to take full control of the drone from my Agent at any time**.

Context: Manual override must allow the human operator to suspend Agent-driven state-changing actions and regain direct control of the system. Manual control should have higher priority than Agent-generated commands.

UI Changes: The interface should clearly show Agent/autonomous control status, manual-control status, and a control for entering manual override.

Extensions / Error States: When manual override is activated, pending Agent commands should be canceled or safely suspended, new state-changing Agent requests should be rejected, the Agent should not automatically regain control, and existing vehicle state must remain synchronized with MAMBA.

Documentation: Document how manual override is activated, what happens to queued Agent commands, and how Agent control may later be restored.

Acceptance Test

You will need:

- MAMBA.
- Operator interface.
- Agent.
- Simulated UAS.

Steps:

1. Begin an Agent-controlled operation.
2. Activate manual override.
3. Attempt another Agent-generated state-changing command.
4. Operate the vehicle manually.
5. Verify control does not automatically return to the Agent.

- Manual override immediately takes precedence.
- Conflicting Agent commands stop or are rejected.
- New Agent commands cannot control the vehicle while override is active.
- Operator can manually control the vehicle.
- Manual-override status is clearly displayed.
- Override event is logged.

Feature F14: Extend Gazebo with MAVLink-M

Prerequisites: F5 — Gazebo Simulation Pipeline; F12 — MAVLink-M Support.

User Story: As a **developer**, I want to **be able to operate a simulated quadcopter with a payload that can be manipulated using MAVLink-M commands in Gazebo using an Agent equipped with MAMBA**.

Context: The existing UAS simulation should be extended with at least one representative MAVLink-M-compatible payload. This environment will allow development of payload tools and safety logic without requiring physical payload hardware.

Extensions / Error States: The simulator should support representative failures such as payload unavailable, invalid payload command, delayed response, unsupported capability, and payload state fault.

Documentation: Document simulated payload capabilities, MAVLink-M configuration, simulation startup procedure, and differences between simulated and physical payload behavior.

Acceptance Test

You will need:

- Gazebo.
- ArduPilot SITL.
- MAVLink-M payload simulation.
- MAMBA.
- MCP-capable Agent.

Steps:

1. Start the MAVLink-M-enabled simulation.
2. Verify the UAS and payload are available.
3. Connect MAMBA.
4. Have the Agent request a supported payload action.
5. Request payload status.
6. Submit an unsupported payload operation.

- Simulated payload is successfully initialized.
- MAMBA can communicate with the payload.
- Agent can invoke a payload operation through MCP.
- MAVLink-M reaches the simulated payload.
- Payload state/status returns to MAMBA.
- Unsupported operations generate controlled errors.

Feature F15: Add State Machines for Payloads in Safety Layer

Prerequisites: F7 — UAV State Machine; F12 — MAVLink-M Support; F14 — MAVLink-M Gazebo Simulation.

User Story: As a **drone operator**, I want to **ensure that my Agent can only use payload tools that can currently be supported by the drone**.

Context: Payload operations may depend upon both the state of the payload and the state of the host vehicle. The safety layer should maintain deterministic payload state and determine whether an Agent-requested payload operation is valid before transmitting MAVLink-M commands. Representative constraints may include payload mode, vehicle state, previous payload operation, authorization state, and payload readiness.

Extensions / Error States:

- Payload state is unknown.
- Payload is disconnected.
- Payload telemetry is stale.
- Requested transition is invalid.
- Vehicle state conflicts with the payload action.
- Authorization is missing.

The operation should fail before transmission when required conditions are not met.

Documentation: Create payload state diagrams identifying states, valid transitions, permitted tools, vehicle-state dependencies, and authorization requirements.

Acceptance Test

You will need:

- MAMBA.
- MAVLink-M Gazebo simulation.
- Defined payload state machine.
- MCP-capable Agent.

Steps:

1. Place the simulated payload in a known state.
2. Request a valid payload operation.
3. Verify execution.
4. Request a payload operation invalid for its current state.
5. Request an operation incompatible with the UAS state.
- Valid payload operations execute.

- Invalid payload transitions are rejected.
- Rejected operations do not reach the payload.
- UAS state dependencies are enforced.
- Agent receives useful rejection information.
- Payload state transitions are logged.

Feature F16: Safety Layer Testing and Benchmarking in Simulation

Prerequisites: F7 — UAV State Machine; F9 — Sequencing and Timing Enforcement; F11 — Human Confirmation and Authorization; F15 — Payload State Machines.

User Story: As a **developer**, I want the **ability to see how the safety layer operates and if it is creating the intended response to Agent tool calls**.

Context: The safety layer should be evaluated against repeatable simulation scenarios to determine whether it correctly permits valid commands and rejects invalid commands. Testing should also collect measurable information such as commands accepted, commands rejected, reason for rejection, safety-layer processing latency, sequence correctness, state-transition correctness, and Agent behavior after receiving a rejection.

Extensions / Error States: The test suite should exercise invalid state transitions, conflicting commands, parallel tool calls, rate-limit violations, missing authorization, stale telemetry, vehicle disconnection, payload failures, and manual override.

Documentation: Maintain a safety test matrix mapping each safety rule to one or more simulation scenarios.

Acceptance Test

You will need:

- Complete MAMBA simulation environment.
- Safety-layer test suite.
- Agent.
- UAS and payload state machines.
- Logging/benchmark collection.

Steps:

1. Execute defined valid command scenarios.
2. Execute invalid state-transition scenarios.

3. Execute concurrent and excessive tool-call scenarios.
4. Exercise human-authorization rules.
5. Exercise payload-state constraints.
6. Record safety decisions and processing latency.
7. Compare observed results with expected behavior.

- Valid operations are allowed.
- Invalid operations are rejected.
- Every rejection identifies the relevant safety reason.
- State transitions occur deterministically.
- Sequencing requirements are enforced.
- Authorization requirements are enforced.
- Safety-layer processing metrics can be collected.
- Required safety simulation tests pass.

Feature F17: Building Out Hardware Testing Pipeline

Prerequisites: F16 — Safety Layer Testing and Benchmarking in Simulation; successful completion of applicable simulation tests.

User Story: As a **drone operator**, I want to **operate a physical drone using natural language mediated by an Agent equipped with MAMBA**.

Context: This feature transitions MAMBA from simulation to physical hardware. Hardware testing should proceed incrementally, beginning with connection and telemetry testing before permitting state-changing autonomous commands.

The hardware environment should preserve the same logical path used in simulation:

Operator → Agent → MAMBA → Safety Layer → MAVSDK → MAVLink/MAVLink-M → Vehicle

Extensions / Error States:

- Incorrect physical vehicle is connected.
- MAVLink connection fails.
- Telemetry is inconsistent.
- Physical configuration differs from simulation.
- Safety configuration is absent.
- Agent attempts an action not approved for hardware testing.
- Communication is lost.

Documentation: Create a hardware-testing procedure including required hardware, firmware versions, connection configuration, pre-test checklist, safety configuration, emergency procedures, test progression, and shutdown procedures.

Acceptance Test

You will need:

- Approved physical ArduPilot-enabled UAS.
- Controlled testing environment.
- MAMBA.
- MCP-capable Agent.
- Operator interface.
- Manual override.
- Required communications hardware.
- Hardware test checklist.

Steps:

1. Complete the pre-test safety checklist.
2. Connect the physical vehicle to MAMBA.
3. Confirm vehicle identity.
4. Verify telemetry-only communication.
5. Submit a natural-language telemetry request.
6. Execute an approved low-risk command.
7. Verify safety-layer enforcement.
8. Verify manual override.

- Physical vehicle connects successfully.
- Vehicle identity is verified before commands are enabled.
- Physical telemetry reaches the Agent through MAMBA.
- Natural-language intent can result in an approved vehicle action.
- Safety-layer constraints remain active.
- Manual override operates correctly.
- Hardware activity is captured in the audit trail.

Feature F18: Add State Machines for USV and UGV in Safety Layer

Prerequisites: F7 — UAV State Machine; F9 — Sequencing and Timing Enforcement; stable safety-layer state-machine abstraction.

User Story: As a **drone operator**, I want the **ability to use USVs and UGVs with MAMBA**.

Context: The safety architecture should support autonomous systems beyond UAVs. MAMBA should allow each vehicle class to define appropriate states and transitions while sharing common infrastructure such as tool registration, authentication, authorization, rate limiting, command sequencing, telemetry handling, and audit logging. A UGV or USV should not be constrained by UAV-specific states such as airborne or landing.

Extensions / Error States:

- Vehicle type cannot be identified.
- Requested tool is not compatible with the vehicle class.
- No state machine exists for the connected platform.
- A transition is invalid for the current platform.
- Platform telemetry required for state determination is unavailable.

Documentation: Provide a USV state diagram, UGV state diagram, common vehicle-state abstraction, and platform-specific tools and restrictions.

Open Questions / Assumptions:

- Determine the minimum USV and UGV state models required for the initial implementation.
- Determine which MCP tools can be shared across UAV, UGV, and USV platforms.
- Determine whether additional vehicle SDK interfaces are necessary beyond MAVSDK.

Acceptance Test

You will need:

- MAMBA.
- Defined USV state machine.
- Defined UGV state machine.
- Appropriate simulated or test vehicle interfaces.

Steps:

1. Initialize MAMBA with a UAV.
2. Verify the UAV state machine is selected.
3. Initialize with a USV.
4. Verify the USV state machine is selected.
5. Initialize with a UGV.
6. Verify the UGV state machine is selected.
7. Execute valid operations for each platform.
8. Attempt platform-incompatible operations.

- Correct state machine is selected for each vehicle class.
- Valid platform-specific transitions execute.
- Invalid transitions are rejected.
- Platform-incompatible tools are rejected before vehicle execution.
- Common safety infrastructure functions across all supported vehicle classes.
- UAV-specific state assumptions are not imposed on USVs or UGVs.

Feature F20: Multi-UAS Management and Command Routing

Prerequisites: F2 — MAVSDK Drone Connection; F3 — Basic Navigation and Telemetry Tools; F7 — UAV State Machine; F9 — Sequencing and Timing Enforcement.

User Story: As a drone operator, I want the ability to manage and issue commands to multiple connected UAS through a single Agent and MAMBA interface.

Context: MAMBA should support simultaneous connections to multiple MAVLink-enabled UAS and maintain an independent representation of each vehicle's connection status, telemetry, and operational state. The MCP server should provide a mechanism for identifying and addressing individual vehicles so that the Agent can direct requests to the appropriate UAS. Each vehicle must remain subject to its own safety-layer state machine, operating constraints, command sequencing, and authorization requirements. The system should allow an operator to request information from multiple vehicles and issue commands to individual vehicles through a shared interface without requiring separate Agent or MAMBA instances for each UAS.

Extensions / Error States:

- Vehicle identifier is missing or invalid.
- Requested vehicle is disconnected or unavailable.
- Multiple vehicles are assigned conflicting identifiers.
- Command is routed to the incorrect vehicle.
- Telemetry from one vehicle is unavailable or outdated.
- A command is valid for one vehicle but invalid for another.
- Multiple commands are issued to different vehicles simultaneously.
- A vehicle disconnects while an operation is in progress.

Documentation: Document how multiple vehicles are discovered, identified, connected, and managed; how MCP tools specify their target vehicle; how individual vehicle states are maintained; and how commands and telemetry are routed between MAMBA and each connected UAS.

Acceptance Test

You will need:

- Running MAMBA server.
- MCP-capable Agent or test client.
- At least two ArduPilot SITL instances.
- MAVSDK.
- Independent vehicle configurations and connection identifiers.

Steps:

1. Start two independent ArduPilot SITL vehicles.
2. Connect both vehicles to MAMBA.
3. Verify that MAMBA identifies and maintains separate connections to both vehicles.
4. Request telemetry from each vehicle independently.
5. Issue a command targeting the first vehicle.
6. Issue a command targeting the second vehicle.
7. Submit a request that retrieves information from both vehicles.
8. Attempt to issue a command using an invalid vehicle identifier.
9. Disconnect one vehicle while maintaining the connection to the other.

Acceptance Criteria:

- MAMBA successfully maintains simultaneous connections to multiple UAS.
- Each vehicle has a unique identifier within MAMBA.
- Telemetry and connection status are maintained independently for each vehicle.
- The Agent can request information from individual vehicles.
- The Agent can direct commands to a specific vehicle.
- Commands are not inadvertently transmitted to another vehicle.
- Each vehicle's safety state machine independently evaluates requested operations.
- Invalid vehicle identifiers generate controlled errors.
- A vehicle disconnection does not interrupt communication with other connected vehicles.
- Simultaneous requests to different vehicles are handled without incorrectly mixing vehicle states or command destinations.